Zero Trust
Konzept und Realität

Michael Gerber
11. November 2022

Agenda

1. Was ist Zero Trust?
2. Verteilte Architektur & Zero Trust Prinzipien
3. Angriffsszenarien und deren Mitigation
4. Fazit

Zero-Trust - sieben Kernprinzipien

Alle Datenquellen und Rechendienste werden als Ressourcen betrachtet.

Der Zugriff auf Ressourcen wird durch eine dynamische Policy bestimmt, diese betrachtet den jeweiligen Kontext der Zugriffsanfrage.

Jegliche Kommunikation wird gesichert, unabhängig vom Standort.

Das Unternehmen überwacht und misst die Integrität und die Sicherheitslage aller eigenen und damit verbundenen Assets.

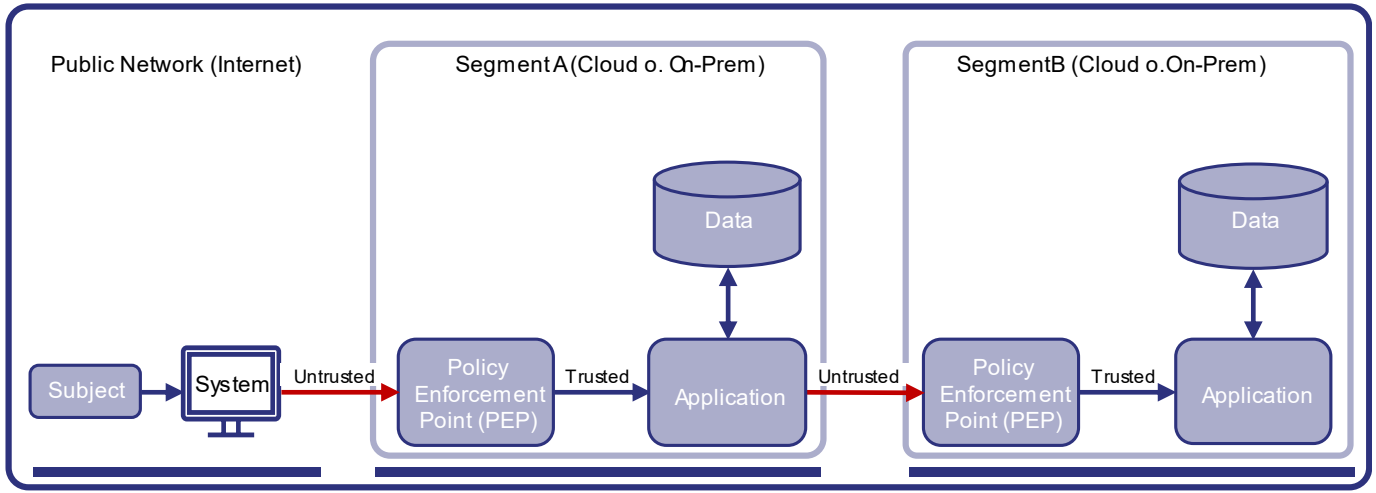
Der Zugriff auf einzelne Ressourcen wird pro Sitzung gewährt.

Alle Ressourcen-Authentifizierung und -Autorisierung sind dynamisch und werden strikt durchgesetzt, bevor der Zugriff gestattet wird.

Das Unternehmen sammelt so viele Informationen wie möglich über den aktuellen Zustand der Assets, Netzwerkinfrastruktur und der Kommunikation und nutzt sie zur Verbesserung seiner Sicherheitslage.

Source: NIST Special Publication 800-207 – Zero Trust Architecture

Verteilte Architektur: (stark) vereinfachte Sicht

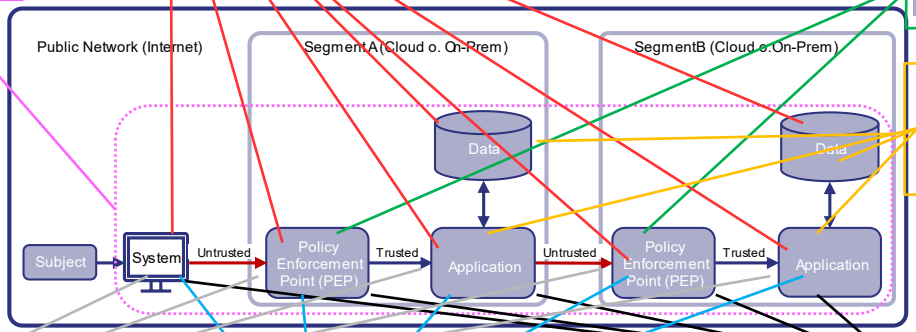


Verteilte Architektur: Verortung der Prinzipien

Das Unternehmen sammelt so viele Informationen wie möglich über den aktuellen Zustand der Assets, Netzwerkinfrastruktur und der Kommunikation und nutzt sie zur Verbesserung seiner Sicherheitslage.

Das Unternehmen überwacht und misst die Integrität und die Sicherheitslage aller eigenen und damit verbundenen Assets.

Der Zugriff auf Ressourcen wird durch eine dynamische Policy bestimmt, diese betrachtet den jeweiligen Kontext der Zugriffsanfrage.



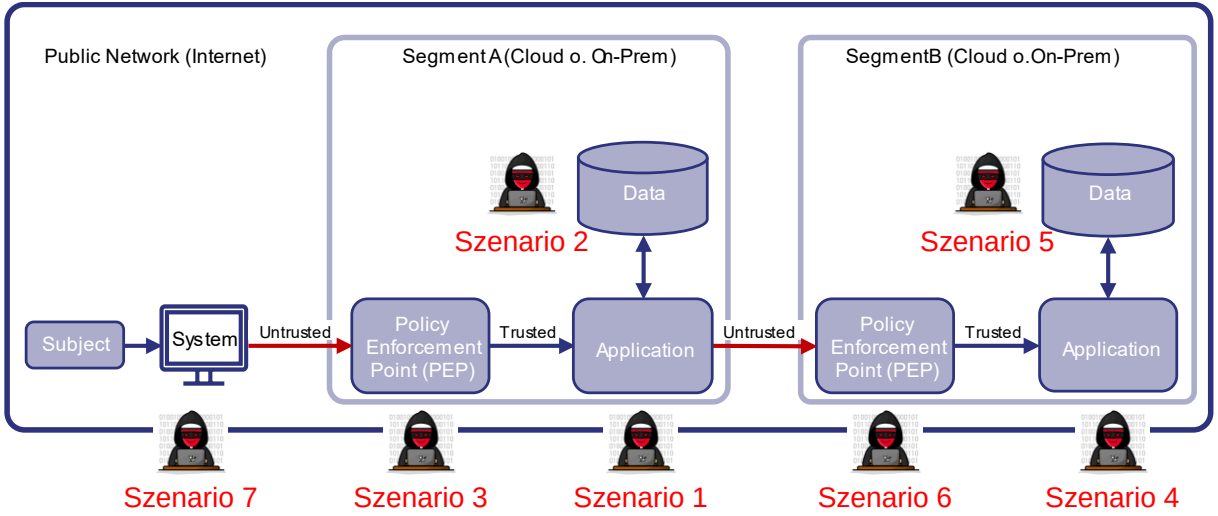
Alle Datenquellen und Rechendienste werden als Ressourcen betrachtet.

Alle Ressourcen-Authentifizierung und -Autorisierung sind dynamisch und werden strikt durchgesetzt, bevor der Zugriff gestattet wird.

Der Zugriff auf einzelne Ressourcen wird pro Sitzung gewährt.

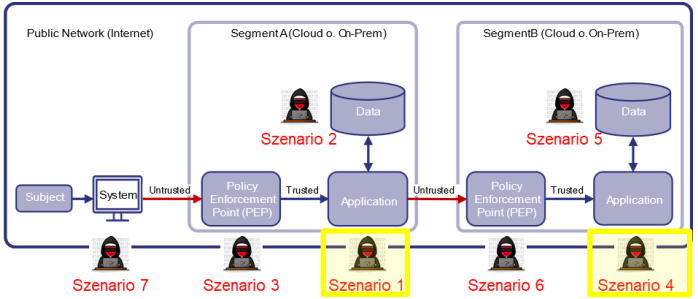
Jegliche Kommunikation wird gesichert, unabhängig vom Standort.

Verteilte Architektur: Angriffsszenarien



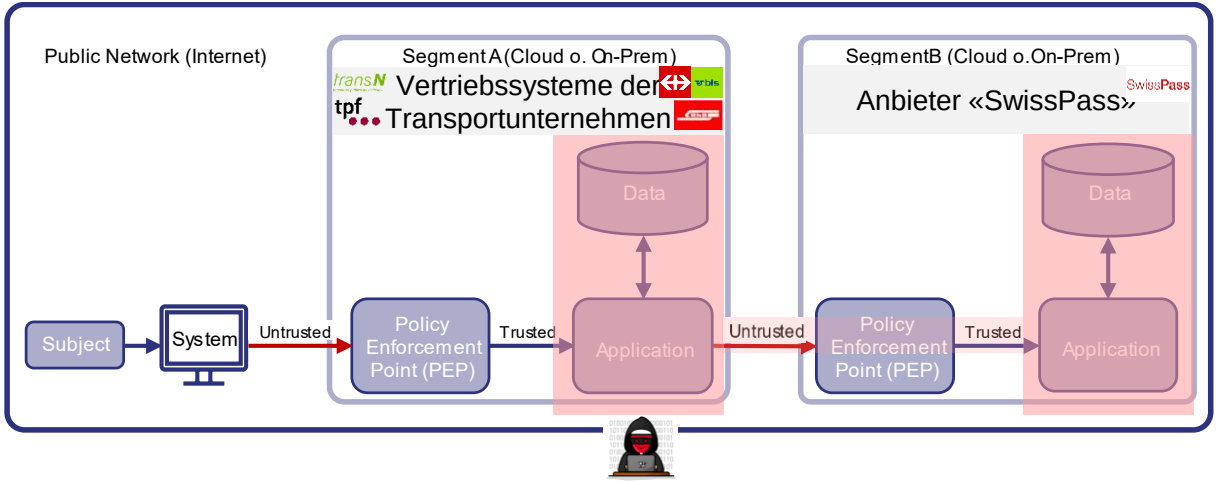
Angreifer

Angriffsszenarien & Gegenmassnahmen



#	Name	Gegenmassnahmen
1;4	Übernahme der Applikation	- Härtung der Applikation - Zugriff einschränken (WAF, Firewall) - Patching (bekannte Sicherheitslücken schliessen) - Limitierung der Anfragen (Benutzer wie auch Administratoren) - Der Zugriff auf die Anwendung erfolgt immer im jeweiligen Nutzerkontext; kein Zugriff mittels Applikationsbenutzer gewähren.
2;5	Übernahme der Datenbank	- Härtung der Datenbank - Verschlüsselung des Inhalts u.a. für Administrationsbenutzer - Limitierung der Anfragen für Administrationsbenutzer
3;6	Übernahme PEP	- Härtung der Anwendung - Zugriff einschränken (WAF, Firewall) - Patching (bekannte Sicherheitslücken schliessen) - Freigabeprozess für Änderungen (Vieraugenprinzip;) - Zugriff nur im Nutzerkontext und auch IP-Adressen einschränken
7	Übernahme Client	- Customer: Empfehlung Kommunizieren - Mitarbeiter: durch das Unternehmen gesteuert & durchsetzen - Partner: Empfehlung Kommunizieren & ggf. durchsetzen

Angriffsszenario 1 - Übernahme der Applikation

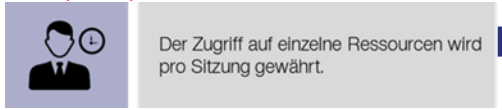


Der Angreifer kann auf alle ■-Daten zugreifen

Angriffsszenario 1: Ein Angreifer kann durch die Übernahme von einer Applikation in einem Segment Zugriff auf alle Daten dieser Applikation und/oder von Daten weiterer Applikationen in anderen Segmenten erhalten.

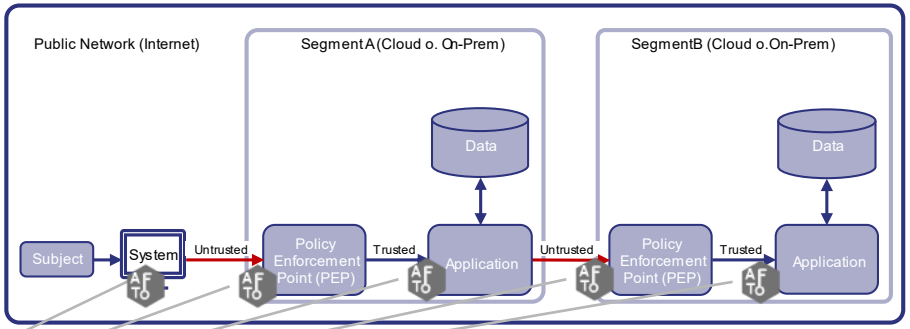
Beschreibung: Ein Angreifer übernimmt die Applikation im Segment A. Er kann alle Daten sehen, die durch diese Applikation fliessen. Dazu gehören natürlich auch die Daten, die von der Applikation im Segment B an die Applikation im Segment A zurückgeliefert werden. Der Angreifer kann auch weitere Daten von der Applikation in Segment B abfragen. Die Zugriffe auf die Applikation in Segment B erfolgen immer im Namen der Applikation in Segment A.

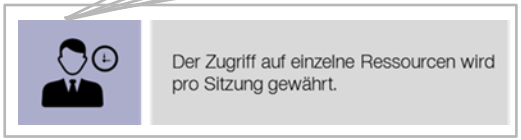
Kernprinzip



Anforderung: Der legitime Zugriff auf Daten soll immer im Kontext des jeweiligen Subjekts respektive dessen Benutzers erfolgen – die entsprechenden Berechtigungen sind diesem zugeordnet.

«Der Zugriff auf einzelne Ressourcen wird pro Sitzung gewährt»
→ Sitzung = Access Token gemäss OAuth2 Protokoll





Access Token von Identity Management zur sitzungsbasierten Zugriffssteuerung

JSON Web Token (JWT) Profile for OAuth 2.0 Access Tokens RFC 9068

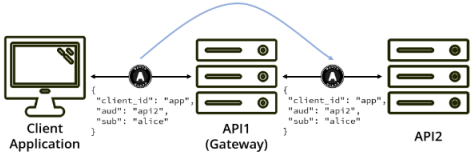
[Access Token] Welches Architektur Pattern erfüllt die Anforderungen?

Der Zugriff auf einzelne Ressourcen wird pro Sitzung gewährt.

https://www.scottbrady91.com/oauth/delegation-patterns-for-oauth-20

Poor-man's Delegation: re-using the access token

The simplest thing to do is have the API gateway re-use the access token it receives and pass it on to the next API.



While this gets the job done, it feels dirty.

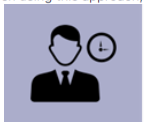
Let's play the authorization request through in our heads. You're asking the user to access a particular API on their behalf, API1 (the API gateway). So API1 is the intended audience of the access token. However, you're now using that token to call API2. This means API2 must accept tokens intended for API1, meaning that the APIs share an audience and scopes, breaking your authorization model.

If you approach it from the other angle and say that API1 must accept tokens from API2, you're again breaking our authorization model, but now our gateway will have to support many different audiences and scopes.

You could maybe mitigate this by saying the token has multiple audiences. However, is API1 even allowed to read the token? For example, this token could be a structured piece of data containing sensitive data that API1 cannot read.

Even then, no matter which way you look at it, you've now opened API2 to being accessed directly. If it is or ever becomes publicly accessible, an access token that should only be used via the gateway can now be used to access API2 directly. You have created the potential to bypass API1.

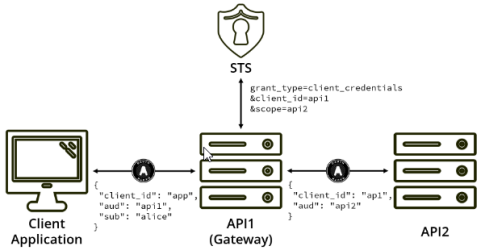
When using this approach, you are not really using delegation but rather impersonation.



Der Zugriff auf einzelne Ressourcen wird pro Sitzung gewährt.

Machine-to-machine authorization using OAuth client credentials

You could have API1 call your authorization server and get a new token using the client credentials grant type. This would mean the initial access token authorized by the user would now be scoped to API1, but unfortunately, it does mean that we lose track of the delegating user.



By getting a new access token using client credentials, you are no longer acting on the user's behalf. In this grant type, the resource owner is the client application (API1), which has an entirely different security profile than the initial user.

To make this approach work, you would have to allow API1 to access any user's data on API2. You would also have to move API2's user-based authorization rules to within API1. If you want to ensure that the user can only access their own data within API2, the only place with enough information to make that decision is now API1.

This *might* work for a high trust scenario, but as soon as you want API2 to be accessible to other applications, you would have to replicate these authorization rules within each consuming application.

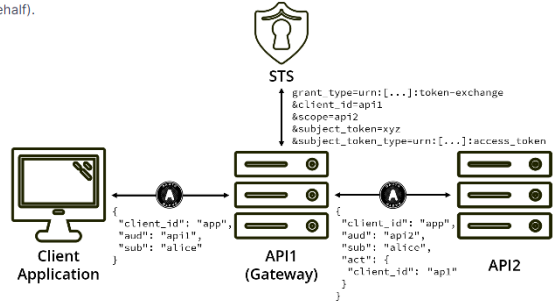
Der Zugriff auf einzelne Ressourcen wird pro Sitzung gewährt.

OAuth 2.0 Token Exchange

The OAuth Working Group has created a specification that formalizes the above delegation scenarios, called token exchange, RFC 8693. It is perfect for API gateways and API-to-API communication while still acting on the user's behalf.

This specification achieves roughly the same as the custom grant above; however, it also takes into account a few other delegation and impersonation scenarios.
The initial token was delegated by `user`, to the client application `app`, and you're still acting on their behalf. But, by using the `act` claim set, you can show that `api1` is the current actor. At the very least, this would be valuable for audit trials.

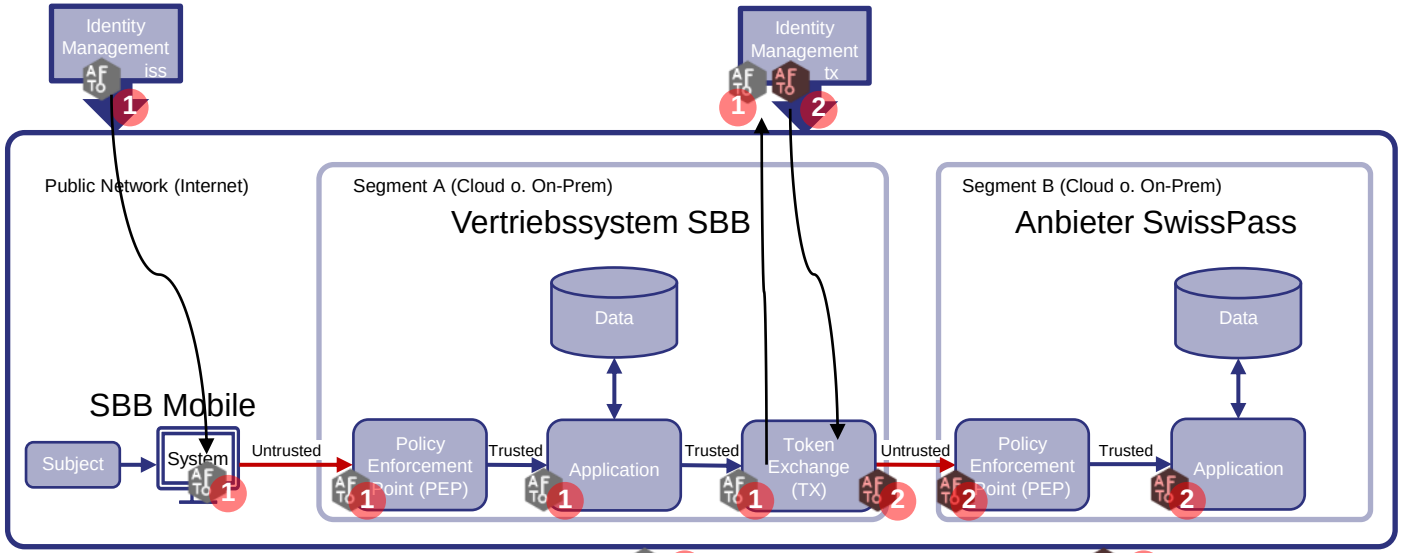
Actor claim sets can be chained (an act claim within an act claim) to present a clear chain of delegation; however, it's always the top-level claims and the direct actor that should be used for authorization policies. The actor claim can also be used for users (e.g. an admin acting on a user's behalf).



- ✓ Erfüllt alle bekannten Anforderungen
- ✓ Ist standardisiert, RFC 8693: OAuth 2.0 Token Exchange

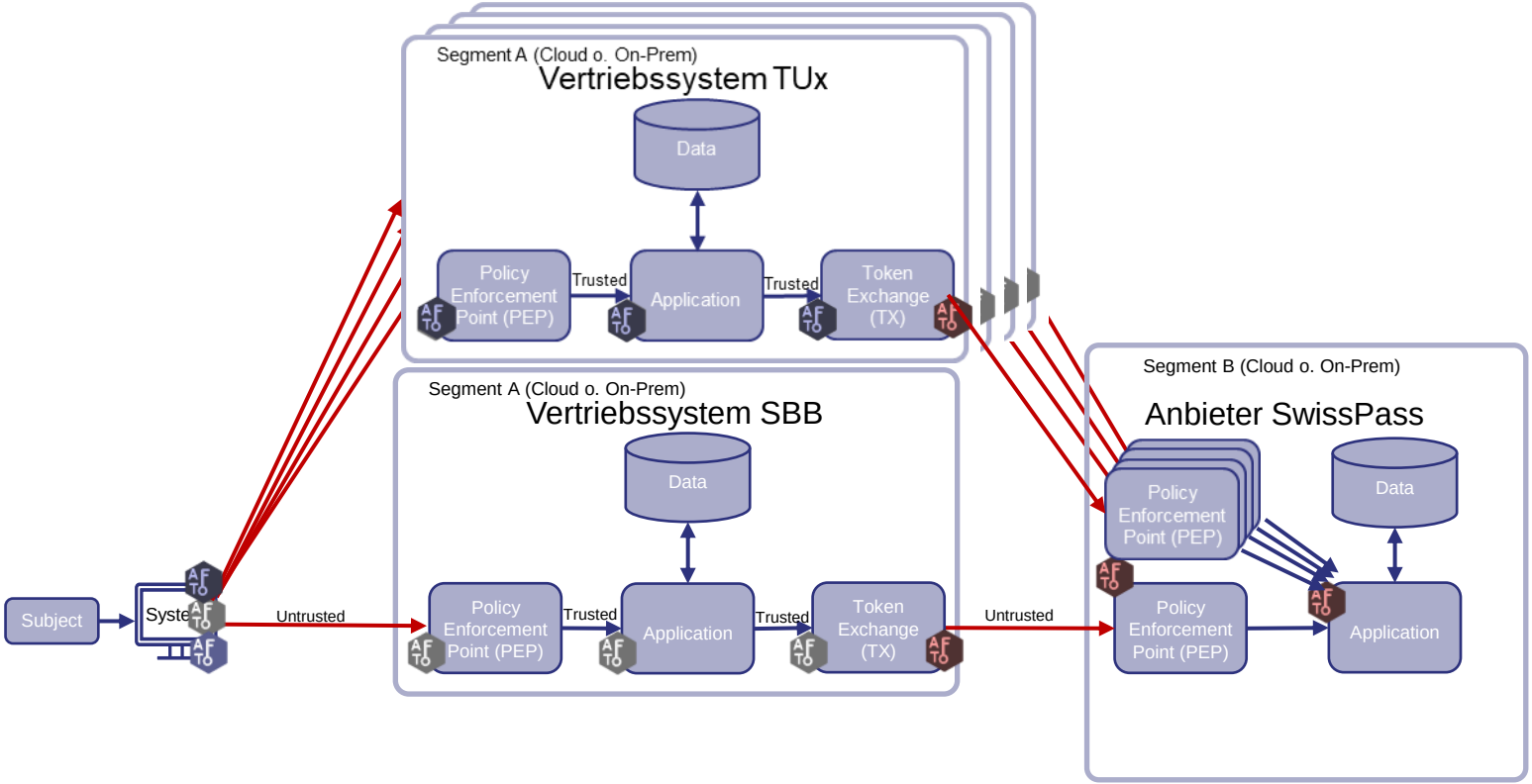
Der Zugriff auf einzelne Ressourcen wird pro Sitzung gewährt.

[Access-Token] Architektur – Token Exchange
Beispiel mit 2 Applikationen (Vertrieb SBB; Anbieter SwissPass)



Fachlich	JWT AT	Inhalt 1	Inhalt 2
Aussteller	iss	OP URL	OP URL Token Exchange
Anwendung	client_id	SBB_Mobile	SBB_Mobile
Inhaber	sub	667345345	1973311412
Empfänger	aud	SBB_Mobile; Vertriebssystem_SBB	Anbieter_SwissPass
Absender	act	-	Vertriebssystem_SBB
Kundennummer	tkid	54656734534	54656734534
Rolle		Kunde	Kunde

[Access-Token] Architektur „SwissPass-Ökosystem“



Zusammenfassung

Das Angriffsszenario „Übernahme der Applikation“
kann unter Anwendung des Zero Trust Prinzip
„Der Zugriff auf einzelne Ressourcen wird pro Sitzung gewährt“
teilweise mitigiert werden.

Technischer Ansatz

→ Access Token

→ Token Exchange

→ PEP

Zero Trust in einem konkreten Fall anwenden heisst:

- Architektur / Anwendungen / Datenflüsse „kennen“
- Relevante Angriffsszenarien finden und risikobasiert bewerten
- Für das Angriffsszenario mit dem höchsten Risiko die mitigierenden Zero Trust Prinzipien finden
- Für die Umsetzung die passende technische Architektur bestimmen

Fragen?